ArchiMate Assessment Tool - Solution Documentation

Version: 1.0.0
Last Updated: November 2024
Author: Development Team
Purpose: Handover documentation for maintenance and future development

---

Table of Contents

1. System Overview and Architecture
2. Technology Stack and Dependencies
3. Setup and Installation
4. Deployment (Replit Autoscale)
5. External Integrations
6. Database Schema and Migrations
7. Authentication and Security
8. Core Features and Workflows
9. API Endpoints
10. Environment Variables and Configuration
11. Testing and Quality Assurance
12. Troubleshooting Common Issues
13. Future Enhancements

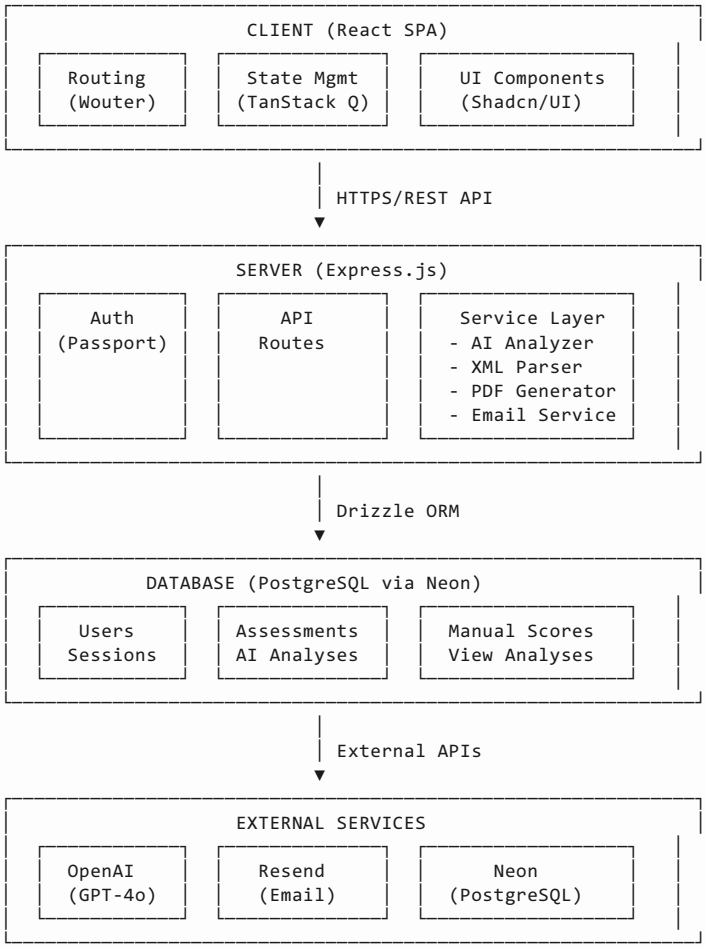
---

1. System Overview and Architecture

1.1 Purpose

The ArchiMate Assessment Tool is a full-stack web application designed to analyze and assess ArchiMate enterprise architecture models. It accepts ArchiMate XML files exported from Bizzdesign, performs AI-powered automated analysis against ArchiMate 3.2 standards, and combines automated scoring with manual expert assessment to generate comprehensive PDF reports.

1.2 High-Level Architecture



1.3 End-to-End Workflow

1. **Upload**: User uploads ArchiMate XML file (max 10MB)
2. **Parse**: XML is parsed to extract elements, relationships, and views
3. **AI Analysis**: Background job analyzes model against ArchiMate 3.2 standards using OpenAI GPT-4o
4. **Manual Assessment**: Assessor reviews AI results and provides manual scores across 5 categories
5. **Finalization**: System calculates combined score (average of AI + manual)
6. **Report Generation**: PDF report generated with AI analysis, manual scores, and digital signatures
7. **Email Notification**: Candidate receives email with report link (optional)

1.4 Core Design Patterns

- **Full-Stack Type Sharing**: Centralized schema definitions in `/shared/schema.ts` using Drizzle ORM and Zod
- **Service Layer Separation**: Business logic organized into dedicated service modules
- **Storage Abstraction**: `IStorage` interface allows flexible storage implementations
- **Component-Based UI**: React components organized by feature and reusability
- **State Management**: TanStack Query for server state, React Hook Form for forms

---

2. Technology Stack and Dependencies

2.1 Frontend

Package	Version	Purpose
React	18.x	UI framework
TypeScript	5.x	Static typing
Vite	5.x	Build tool and dev server
Wouter	3.x	Client-side routing
TanStack Query	5.x	Server state management
React Hook Form	7.x	Form state and validation
Zod	3.x	Schema validation
Shadcn/UI	Latest	Component library (Radix UI + Tailwind)
Tailwind CSS	3.x	Utility-first CSS
Lucide React	Latest	Icon library
date-fns	3.x	Date manipulation

2.2 Backend

Package	Version	Purpose
Express	4.x	Web server framework
TypeScript	5.x	Static typing
tsx	4.x	TypeScript execution for dev
Passport.js	0.7.x	Authentication middleware
passport-local	1.x	Username/password strategy
bcrypt	5.x	Password hashing
express-session	1.x	Session management
connect-pg-simple	9.x	PostgreSQL session store
Drizzle ORM	0.33.x	Type-safe database toolkit
@neondatabase/serverless	Latest	Neon PostgreSQL driver
Multer	1.x	File upload handling
xml2js	0.6.x	XML parsing
PDFKit	0.15.x	PDF generation
OpenAI SDK	4.x	AI analysis integration
Resend	4.x	Email delivery (optional)

2.3 Development Tools

Tool	Purpose
drizzle-kit	Database migrations and introspection
esbuild	Production backend bundling
@replit/vite-plugin-*	Replit development environment integration

---

3. Setup and Installation

3.1 Prerequisites

- Node.js 20.x or higher
- PostgreSQL database (Neon serverless recommended)
- OpenAI API key with GPT-4o access
- (Optional) Resend API key or SMTP credentials for email

3.2 Initial Setup

1. **Clone Repository**

   ```
   # Already on Replit - no cloning needed
   ```

2. **Install Dependencies**

   ```
   npm install
   ```

3. **Configure Environment Variables**

   Create/edit `.env` file with required variables (see Section 10):

   ```
   DATABASE_URL=postgresql://user:password@host/database
   SESSION_SECRET=your-secure-random-string-min-32-chars
   OPENAI_API_KEY=sk-...
   ```

4. **Initialize Database**

   ```
   # Push schema to database (creates tables)
   npm run db:push
   ```

5. **Start Development Server**

   ```
   npm run dev
   ```

   The application will:

   - Start Express server on port 5000
   - Start Vite dev server with HMR
   - Auto-create super admin account (username: `admin`, password: `Admin@123`)
   - Open at `http://localhost:5000`

3.3 First Login

1. Navigate to `/login`
2. Login with super admin credentials:
   - Username: `admin`
   - Password: `Admin@123`
3. **IMPORTANT**: Change password immediately on first login
4. Create assessor accounts via Admin panel (`/admin`)

---

4. Deployment (Replit Autoscale)

4.1 Deployment Overview

The application is deployed to **Replit Autoscale**, which provides: - Automatic horizontal scaling based on traffic - SSL/TLS termination - Health check monitoring - Zero-downtime deployments - PostgreSQL session persistence across instances

4.2 Production Build

Replit automatically runs the build process on deployment:

```
npm run build
```

This executes: 1. **Frontend Build**: Vite builds React app to `dist/public` 2. **Backend Build**: esbuild bundles Express server to `dist/index.js` 3. **Database Migration**: `npm run db:push` creates production tables

4.3 Production Start

```
npm start
```

Starts the production server which: - Serves bundled Express app from `dist/index.js` -

Serves static frontend from `dist/public` - Detects `REPLIT_DEPLOYMENT=1` environment variable - Uses `PRODUCTION_DATABASE_URL` or falls back to `DATABASE_URL`

4.4 Critical Deployment Configuration

Trust Proxy Setting

File: `server/index.ts`

```
// CRITICAL: Trust Replit's reverse proxy for secure cookies
app.set('trust proxy', 1);
```

Why it's needed: Replit's Autoscale uses an SSL-terminating load balancer. Without this setting, Express won't recognize HTTPS connections and secure cookies will fail.

Session Store Configuration

File: `server/index.ts`

```
app.use(session({
  store: new pgSession({
    pool: db,
    tableName: 'session',
    createTableIfMissing: true
  }),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: process.env.NODE_ENV === 'production',
    httpOnly: true,
    maxAge: 24 * 60 * 60 * 1000,
    sameSite: 'lax'
  }
}));
```

Why PostgreSQL session store: Autoscale can run multiple instances. PostgreSQL-backed sessions ensure users stay authenticated across instance switches.

Health Check Endpoint

File: `server/routes.ts`

```
app.get("/health", (_req, res) => {
  res.status(200).json({ status: "healthy" });
});
```

Purpose: Autoscale pings this endpoint (responds in ~3-5ms) to determine instance readiness. No database operations to ensure instant response.

Server Configuration

File: `server/index.ts`

```
const server = app.listen(PORT, "0.0.0.0", {
  reusePort: true
});
```

Why `reusePort: true`: Enables multiple instances to bind to the same port for horizontal scaling.

4.5 Admin Account Seeding

File: `server/seed-admin.ts`

On startup, the application: 1. Waits 100ms (ensures health checks pass first) 2. Checks if super admin exists 3. Creates default admin if missing 4. Logs result without blocking startup

Important: If production database is reset, super admin will be auto-recreated on next deployment.

4.6 Environment Variables (Production)

Required in Replit Secrets:

```
DATABASE_URL=postgresql://...           # Auto-provided by Replit
PRODUCTION_DATABASE_URL=postgresql://... # Optional separate prod DB
SESSION_SECRET=minimum-32-character-random-string
OPENAI_API_KEY=sk-...
```

Optional:

```
RESEND_API_KEY=re_...                   # For email notifications
SMTP_HOST=smtp.example.com              # Alternative to Resend
SMTP_PORT=587
SMTP_USER=user@example.com
SMTP_PASSWORD=...
SMTP_FROM=noreply@example.com
FRONTEND_URL=https://your-app.replit.app
```

4.7 Deployment Checklist

- ☐ All environment variables configured in Replit Secrets
- ☐ `SESSION_SECRET` is secure (min 32 chars, random)
- ☐ OpenAI API key has sufficient credits
- ☐ Database connection string is correct
- ☐ Test login after deployment
- ☐ Verify file upload works
- ☐ Test complete assessment workflow
- ☐ Check email notifications (if enabled)
- ☐ Monitor logs for errors

4.8 Post-Deployment Verification

1. **Check Health**: `https://your-app.replit.app/health` → `{"status":"healthy"}`
2. **Test Login**: Navigate to `/login` and authenticate
3. **Upload Test File**: Create assessment with sample XML
4. **Monitor Logs**: Check for startup errors or warnings

---

5. External Integrations

5.1 OpenAI API Integration

5.1.1 Overview

Service: `server/services/ai-analyzer.ts`
Model: GPT-4o (configurable via environment)
Purpose: Automated ArchiMate model analysis against industry standards

5.1.2 Configuration

```
// Environment Variables
OPENAI_API_KEY=sk-proj-...              # Required
OPENAI_MODEL=gpt-4o                     # Default: gpt-4o
OPENAI_MAX_TOKENS=16000                 # Default: 16000
OPENAI_TEMPERATURE=0                    # Deterministic output
```

5.1.3 Analysis Process

1. **XML Summarization**: Large files (>20k tokens) are automatically summarized
   - Extracts essential ArchiMate data (elements, relationships, views)
   - Removes verbose positioning/styling data
   - Achieves ~70% size reduction
   - Prevents OpenAI token limit errors
2. **Streaming Analysis**: Uses OpenAI streaming API for real-time progress

   ```
   const stream = await openai.chat.completions.create({
     model: this.model,
     messages: [...],
     temperature: 0,
     seed: 42,  // Deterministic results
     stream: true
   });
   ```

3. **Retry Logic**: Automatic retry on transient failures
   - Rate limit errors (429): Returns fallback analysis
   - Server errors (5xx): Returns fallback analysis
   - Network errors: Throws exception
4. **Scoring System**:
   - Evaluates 5 categories with weighted scores
   - RAG status (Red/Amber/Green) based on thresholds
   - Penalties for insufficient views (expected 6 views)
   - Issues tracked with severity, type, and affected views

5.1.4 Rate Limits and Costs

OpenAI Pricing (as of 2024): - GPT-4o: ~$5 per 1M input tokens, ~$15 per 1M output tokens - Typical assessment: ~15k input tokens, ~5k output tokens - Estimated cost per assessment: ~$0.15-0.25

Rate Limits: - Tier 1: 500 requests/day, 10k tokens/minute - Tier 2: Higher limits with payment history

Best Practices: - Monitor OpenAI dashboard for usage - Set up billing alerts - Implement usage quotas if needed

5.1.5 Error Handling

```
try {
  const analysis = await aiAnalyzer.analyzeModel(parsedXml);
} catch (error) {
  if (error.status === 429) {
    // Rate limit - use fallback
  } else if (error.status >= 500) {
    // OpenAI service error - use fallback
  } else {
    // Fatal error - notify user
  }
}
```

Fallback Analysis: Returns minimal valid analysis with: - Default scores (50%) - Generic recommendations - Error indication in comments

5.2 Email Integration (Resend/SMTP)

5.2.1 Overview

Service: `server/services/email-service.ts`
Purpose: Send assessment completion notifications to candidates

5.2.2 Resend API (Recommended)

```
# Environment Variables
RESEND_API_KEY=re_...
SMTP_FROM=noreply@yourdomain.com
FRONTEND_URL=https://your-app.replit.app
```

Features: - Simple REST API - High deliverability - Generous free tier (100 emails/day) - Detailed analytics

Example Email:

```
await resend.emails.send({
  from: process.env.SMTP_FROM!,
  to: candidateEmail,
  subject: `ArchiMate Assessment Complete - ${candidateName}`,
  html: emailTemplate
});
```

5.2.3 SMTP Fallback

```
# Environment Variables
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-app-password
SMTP_FROM=noreply@yourdomain.com
FRONTEND_URL=https://your-app.replit.app
```

Note: SMTP implementation is available as fallback if Resend is not configured.

5.2.4 Email Templates

Email includes: - Candidate name and role - Overall assessment score - Grade (Outstanding/Average/Poor) - Link to download PDF report - Assessor signature (if provided)

5.3 Neon Database Integration

5.3.1 Overview

Provider: Neon (Serverless PostgreSQL)
Connection: `@neondatabase/serverless` driver
ORM: Drizzle

5.3.2 Connection Configuration

```
import { neon } from "@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-http";

const sql = neon(connectionString);
const db = drizzle(sql, { schema });
```

Benefits: - Serverless (auto-pause when idle) - HTTP connection (works in serverless environments) - Branch-based development databases - Automatic backups

5.3.3 Database Selection Logic

```
const isProduction = process.env.REPLIT_DEPLOYMENT === "1";
const connectionString = isProduction && process.env.PRODUCTION_DATABASE_U
  ? process.env.PRODUCTION_DATABASE_URL
  : process.env.DATABASE_URL;
```

Development: Uses `DATABASE_URL` (shared database)
Production: Uses `PRODUCTION_DATABASE_URL` if available, else `DATABASE_URL`

---

6. Database Schema and Migrations

6.1 Schema Overview

All tables defined in: `shared/schema.ts`

6.1.1 Users Table

```
export const users = pgTable("users", {
  id: varchar("id").primaryKey().$defaultFn(() => nanoid()),
  username: varchar("username", { length: 255 }).notNull().unique(),
  password: varchar("password", { length: 255 }).notNull(),
  displayName: varchar("display_name", { length: 255 }).notNull(),
  email: varchar("email", { length: 255 }),
  role: varchar("role", { length: 50 }).notNull().default("assessor"),
  forcePasswordChange: boolean("force_password_change").notNull().default(
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
});
```

Roles: - `super_admin`: Full system access, user management - `assessor`: Standard user, can create/complete assessments

6.1.2 Assessments Table

```
export const assessments = pgTable("assessments", {
  id: varchar("id").primaryKey().$defaultFn(() => nanoid()),
  candidateName: varchar("candidate_name", { length: 255 }).notNull(),
  candidateEmail: varchar("candidate_email", { length: 255 }),
  candidateRole: varchar("candidate_role", { length: 255 }).notNull(),
  assessorName: varchar("assessor_name", { length: 255 }).notNull(),
  assessmentDate: timestamp("assessment_date").notNull(),
  candidateCompletionDate: timestamp("candidate_completion_date"),

  xmlContent: text("xml_content").notNull(),

  aiScore: numeric("ai_score", { precision: 5, scale: 2 }).default('0'),
  manualScore: numeric("manual_score", { precision: 5, scale: 2 }).default
  overallScore: numeric("overall_score", { precision: 5, scale: 2 }).defau

  status: varchar("status", { length: 50 }).notNull().default('pending'),
  grade: varchar("grade", { length: 50 }),

  assessorSignature: text("assessor_signature"),
  assessorSignatureDate: timestamp("assessor_signature_date"),
  reviewerSignature: text("reviewer_signature"),
  reviewerSignatureDate: timestamp("reviewer_signature_date"),
  candidateSignature: text("candidate_signature"),
  candidateSignatureDate: timestamp("candidate_signature_date"),

  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
});
```

Status Flow: 1. `pending`: Initial state (XML uploaded, no AI analysis yet) 2. `analyzing`: AI analysis in progress 3. `analyzed`: AI complete, awaiting manual assessment 4. `completed`: Manual assessment finalized

6.1.3 AI Analyses Table

```
export const aiAnalyses = pgTable("ai_analyses", {
  id: varchar("id").primaryKey().$defaultFn(() => nanoid()),
  assessmentId: varchar("assessment_id").notNull().references(() => assess
  categoryName: varchar("category_name", { length: 255 }).notNull(),
  categoryWeight: numeric("category_weight", { precision: 5, scale: 2 }).n
  score: numeric("score", { precision: 5, scale: 2 }).notNull(),
  maxScore: numeric("max_score", { precision: 5, scale: 2 }).notNull(),
  ragStatus: varchar("rag_status", { length: 50 }).notNull(),
  issues: jsonb("issues").notNull().default([]),
  recommendations: jsonb("recommendations").notNull().default([]),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});
```

Categories (5 total): 1. Meta-model & Relationships (25% weight) 2. Scenario Interpretation (20% weight) 3. Viewpoints & Stakeholder Relevance (20% weight) 4. Cross-layer Alignment (20% weight) 5. Visual Clarity (15% weight)

6.1.4 Manual Scores Table

```
export const manualScores = pgTable("manual_scores", {
  id: varchar("id").primaryKey().$defaultFn(() => nanoid()),
  assessmentId: varchar("assessment_id").notNull().references(() => assessi
  categoryName: varchar("category_name", { length: 255 }).notNull(),
  subCategoryName: varchar("sub_category_name", { length: 255 }).notNull()
  weight: numeric("weight", { precision: 5, scale: 2 }).notNull(),
  rating: numeric("rating", { precision: 3, scale: 2 }),
  weightedScore: numeric("weighted_score", { precision: 5, scale: 2 }).def
  comments: text("comments"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
});
```

Rating Scale: - 0: Missing - 1: Poor (20%) - 2: Below Average (40%) - 3: Adequate (60%) - 4: Good (80%) - 5: Outstanding (100%)

Weighted Score Calculation:

```
weightedScore = (rating / 5) × weight × 100
```

6.1.5 View Analyses Table

```
export const viewAnalyses = pgTable("view_analyses", {
  id: varchar("id").primaryKey().$defaultFn(() => nanoid()),
  assessmentId: varchar("assessment_id").notNull().references(() => assessi
  viewName: varchar("view_name", { length: 255 }).notNull(),
  viewType: varchar("view_type", { length: 255 }).notNull(),
  layer: varchar("layer", { length: 100 }).notNull(),
  elementCount: integer("element_count").notNull().default(0),
  issueCount: integer("issue_count").notNull().default(0),
  severity: varchar("severity", { length: 50 }).notNull(),
  issues: jsonb("issues").notNull().default([]),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});
```

6.1.6 Session Table

Auto-created by `connect-pg-simple`:

```
CREATE TABLE "session" (
  "sid" varchar NOT NULL PRIMARY KEY,
  "sess" json NOT NULL,
  "expire" timestamp(6) NOT NULL
);
```

6.2 Relationships

```
users
  └── (assessor creates) → assessments
                              ├── aiAnalyses (1-to-many)
                              ├── manualScores (1-to-many)
                              └── viewAnalyses (1-to-many)
```

6.3 Migration Workflow

IMPORTANT: Never manually write SQL migrations. Use Drizzle Kit.

6.3.1 Schema Changes

1. **Edit Schema**: Modify `shared/schema.ts`
2. **Push to Database**:

   ```
   npm run db:push
   ```

3. **Force Push** (if data-loss warning):

   ```
   npm run db:push --force
   ```

6.3.2 Critical Rules

- **NEVER change primary key ID types** (e.g., `serial` ↔ `varchar`)
- Match Drizzle schema to existing database structure
- Test schema changes in development first
- Backup production database before major migrations

6.3.3 Production Migrations

For production database on Replit: 1. Changes are applied during deployment build step 2. `npm run build` includes `npm run db:push` 3. Monitor deployment logs for migration errors

---

7. Authentication and Security

7.1 Authentication Flow

7.1.1 Strategy: Passport Local

File: `server/auth.ts`

```
passport.use(new LocalStrategy(async (username, password, done) => {
  const user = await storage.getUserByUsername(username);
  if (!user) return done(null, false, { message: "Invalid credentials" });
  if (!user.isActive) return done(null, false, { message: "Account disable

  const isValid = await bcrypt.compare(password, user.password);
  if (!isValid) return done(null, false, { message: "Invalid credentials"

  done(null, user);
}));
```

7.1.2 Password Security

- **Hashing**: bcrypt with 10 rounds
- **Minimum Length**: 8 characters
- **Complexity**: No requirements (to be added if needed)
- **Storage**: Hashed passwords only, never plaintext

Password Hashing:

```
const hashedPassword = await bcrypt.hash(password, 10);
```

7.1.3 Forced Password Change

New users created by super admin: 1. Receive auto-generated password (12 chars, crypto-random) 2. `forcePasswordChange` flag set to `true` 3. Must change password on first login 4. Cannot access protected routes until password changed

Implementation:

```
if (user.forcePasswordChange) {
  return <Redirect to="/login" />;
}
```

7.2 Session Management

7.2.1 Configuration

```
app.use(session({
  store: new pgSession({
    pool: db,
    tableName: 'session',
    createTableIfMissing: true
  }),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: process.env.NODE_ENV === 'production',
    httpOnly: true,
    maxAge: 24 * 60 * 60 * 1000, // 24 hours
    sameSite: 'lax'
  }
}));
```

7.2.2 Session Security

- **Storage**: PostgreSQL-backed (not memory or cookies)
- **HttpOnly**: Cookies not accessible via JavaScript
- **Secure**: HTTPS-only in production
- **SameSite**: CSRF protection
- **Expiration**: 24 hours
- **Trust Proxy**: Required for Autoscale (see Section 4.4)

7.3 Route Protection

7.3.1 Middleware

Authentication Required:

```
function requireAuth(req: Request, res: Response, next: NextFunction) {
  if (!req.isAuthenticated()) {
    return res.status(401).json({ message: "Not authenticated" });
  }
  next();
}
```

Role Required:

```
function requireRole(role: string) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.isAuthenticated()) {
      return res.status(401).json({ message: "Not authenticated" });
    }
    if (req.user!.role !== role) {
      return res.status(403).json({ message: "Access denied" });
    }
    next();
  };
}
```

7.3.2 Protected Routes

Public Routes: - `/login` - Authentication page - `/health` - Health check (no auth)

Authenticated Routes (require login): - `/` - New assessment - `/records` - Assessment records - `/assessment/:id` - Assessment details

Admin Routes (require `super_admin` role): - `/admin` - User management

7.4 File Upload Security

Validation:

```
const upload = multer({
  storage: multer.memoryStorage(),
  limits: {
    fileSize: 10 * 1024 * 1024, // 10MB
  },
  fileFilter: (_req, file, cb) => {
    if (file.mimetype === 'text/xml' || file.mimetype === 'application/xml
      cb(null, true);
    } else {
      cb(new Error('Only XML files are allowed'));
    }
  }
});
```

Security Measures: - File type validation (XML only) - Size limit (10MB max) - Memory storage (not disk) - Content validation (XML parsing)

7.5 Environment Variable Validation

Critical Check on Startup:

```
if (process.env.NODE_ENV === 'production' && !process.env.SESSION_SECRET)
  console.error('FATAL: SESSION_SECRET must be set in production');
  process.exit(1);
}
```

Why it's critical: Without `SESSION_SECRET`, sessions are insecure and production deployment should fail.

7.6 Security Best Practices Implemented

- Password hashing with bcrypt
- HTTP-only secure session cookies
- PostgreSQL session persistence
- Role-based access control
- File upload validation
- Environment variable validation
- Trust proxy for SSL termination
- CSRF protection (SameSite cookies)
- No secrets in logs or responses

7.7 Security Recommendations

- [ ] Add password complexity requirements (uppercase, numbers, symbols)
- [ ] Implement rate limiting on login endpoint
- [ ] Add 2FA for super admin accounts
- [ ] Implement session audit logging
- [ ] Add Content Security Policy headers
- [ ] Implement XSS protection headers
- [ ] Add database query parameterization audit
- [ ] Regular security dependency updates

---

8. Core Features and Workflows

8.1 User Authentication

8.1.1 Login Flow

1. User navigates to `/login`
2. Enters username and password
3. System validates credentials
4. If `forcePasswordChange === true`:
   - Show password change form
   - Require new password (min 8 chars)
   - Update password and set `forcePasswordChange = false`
5. Redirect to dashboard (`/`)

8.1.2 Password Change (First Login)

Form Fields: - Current password (temporary password) - New password (min 8 characters) - Confirm new password

Validation: - Current password must match stored hash - New password must meet requirements - Confirm password must match new password

8.2 Assessment Creation and Analysis

8.2.1 XML Upload Workflow

Page: `/` (New Assessment Dashboard)

1. **File Selection**:
   - Drag & drop XML file
   - Or click to browse files
   - Accepts `.xml` files only (max 10MB)
2. **File Validation**:

```
// Client-side validation
accept: { 'text/xml': ['.xml'], 'application/xml': ['.xml'] }
maxSize: 10 * 1024 * 1024 // 10MB

// Server-side validation
if (!file.mimetype.includes('xml')) {
  throw new Error('Only XML files allowed');
}
```

3. **Upload Process**:
   - File sent to `/api/assessments` (POST)
   - Server receives multipart form data
   - XML stored in `assessments.xmlContent`
   - Assessment ID returned to client

8.2.2 AI Analysis Pipeline

Background Process (does not block upload response):

1. **Status Update**: Set assessment status to `analyzing`
2. **XML Parsing** (`server/services/xml-parser.ts`):

```
const parsed = await parseXML(xmlContent);
// Extracts: elements, relationships, views, properties
```

3. **AI Analysis** (`server/services/ai-analyzer.ts`):
   - Check file size (if >20k tokens, summarize first)
   - Call OpenAI API with streaming
   - Parse JSON response with 5 category analyses
   - Store results in `aiAnalyses` table
   - Extract view-specific data to `viewAnalyses` table

4. **Score Calculation**:

```
aiScore = sum(category.score) / sum(category.maxScore) * 100
```

5. **Manual Scores Creation**:
   - Create 20 manual score entries (4 subcategories × 5 categories)
   - All initially `null` (awaiting assessor input)
6. **Status Update**: Set assessment status to `analyzed`
7. **Next Steps Alert**: UI shows guidance to move to Manual Assessment tab

8.2.3 Error Handling

Scenarios: - XML parsing failure → User-friendly error message - OpenAI timeout → Fallback analysis with default scores - Rate limit exceeded → Fallback analysis with quota error - Network error → Retry logic or fallback

8.3 Manual Assessment

8.3.1 Assessment Details Page

Route: `/assessment/:id`

Tabs: 1. **Automated Analysis**: AI results (4 sub-tabs) 2. **Manual Assessment**: Assessor scoring interface 3. **Detailed Report**: Complete breakdown 4. **View Analysis**: View-specific issues

8.3.2 Manual Scoring Interface

Candidate Information: - Name (required) - Email (optional, for notification) - Role (required, dropdown) - Completion Date (auto-filled, editable)

Scoring Table:

Category	Subcategory	Weight	AI Rating	Manual Rating	Weighted Score	Comments
Meta-model	Correctness	10%	Good (4)	[Dropdown 0-5]	Auto-calc	[Text]
…	…	…	…	…	…	…

Rating Scale: - 0 (0%): Missing - 1 (20%): Poor - 2 (40%): Below Average - 3 (60%): Adequate - 4 (80%): Good - 5 (100%): Outstanding

AI Rating Display: - Shows AI's category score as descriptive word - Helps inform manual assessment - Not editable (reference only)

Auto-Save: - Ratings saved immediately on change - Comments auto-saved on blur or manual save - Optimistic updates with rollback on error

8.3.3 Digital Signatures

Three Signature Types: 1. **Assessor Signature**: Person who conducted assessment 2. **Reviewer Signature**: Optional quality assurance reviewer 3. **Candidate Signature**: Person being assessed (rarely used)

Signature Process: 1. Click signature pad 2. Draw signature with mouse/touch 3. **IMPORTANT**: Click "Save Signature" button 4. Signature stored as base64 PNG 5. Date automatically recorded

Common Issue: Signatures won't appear in PDF if "Save Signature" button not clicked.

8.3.4 Finalization

Requirements Before Finalization: - All candidate fields filled - All 20 manual ratings provided (no null values) - At least one signature (assessor recommended)

Finalization Process: 1. Click "Complete Assessment" button 2. System validates requirements 3. Calculates scores: `typescript manualScore = sum(weightedScores) overallScore = (aiScore + manualScore) / 2 grade = overallScore >= 80 ? 'Outstanding' : overallScore >= 60 ? 'Average' : 'Poor'` 4. Updates assessment status to `completed` 5. Sends email notification (if candidate email provided) 6. Assessment locked (no further edits)

8.4 Report Generation

8.4.1 PDF Report

Trigger: Click "Download Report" button in Detailed Report tab

Endpoint: `POST /api/assessments/:id/report`

Report Contents: 1. **Cover Page**: - ArchiMate Assessment Report title - Candidate name and role - Assessment date - Overall score and grade

2. **AI Analysis Section**:
   - Score breakdown by category
   - RAG status indicators
   - Issues found (grouped by category)
   - Recommendations
3. **Manual Assessment Section**:
   - Scoring table with all subcategories
   - Assessor comments
   - Weighted scores
4. **Combined Scoring**:
   - AI Score: X%
   - Manual Score: Y%
   - Overall Score: Z%
   - Grade: Outstanding/Average/Poor
5. **Signatures Section**:
   - Assessor signature + date
   - Reviewer signature + date (if provided)
   - Candidate signature + date (if provided)

File Naming:

```
ArchiMate_Assessment_[CandidateName]_[Date].pdf
Example: ArchiMate_Assessment_John_Doe_2024-11-20.pdf
```

Technical Implementation (`server/services/pdf-generator.ts`): - Uses PDFKit library - Generates PDF in memory (not saved to disk) - Embeds base64 signature images - Returns PDF as buffer - Client receives as blob download

8.5 Admin User Management

8.5.1 User Management Page

Route: `/admin` (super_admin only)

Features: - List all users (display name, username, role, status) - Create new assessor accounts - Edit user display names - Toggle user active/inactive status - Delete assessor accounts (cannot delete super admins) - Reset user passwords

8.5.2 Create Assessor Workflow

1. **Click “Create User” button**
2. **Fill form**:
   - Username (unique, required)
   - Display Name (required)
   - Email (optional)
3. **Submit**
4. **System generates**:
   - Random secure password (12 chars, crypto-random)
   - Sets `forcePasswordChange = true`
5. **Password displayed ONCE**:
   - Admin must copy and share with new user
   - Password never shown again
6. **New user’s first login**:
   - Must change temporary password
   - Cannot access app until password changed

8.5.3 Password Reset

Scenario: User forgot password or needs new temporary password

1. Admin clicks “Reset Password” for user
2. System generates new temporary password
3. Sets `forcePasswordChange = true`
4. Password displayed ONCE to admin
5. Admin shares with user
6. User must change on next login

8.6 Assessment Records

8.6.1 Records Page

Route: `/records`

Display: - Table of all assessments - Columns: Candidate, Role, Date, Status, Overall Score, Actions

Status Badges: - **Pending**: Yellow (draft, in progress) - **Analyzing**: Blue (AI analysis running) - **Analyzed**: Purple (awaiting manual scoring) - **Completed**: Green (finalized)

Actions: - **View Details**: Navigate to assessment page - **Download Report**: Generate PDF (completed only) - **Delete**: Remove assessment (with confirmation)

Filtering/Sorting: - Sort by date (newest first) - Filter by status (to be implemented)

---

9. API Endpoints

9.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	`/api/auth/login`	No	Login with username/password
POST	`/api/auth/logout`	Yes	End session
GET	`/api/auth/session`	No	Get current user session
POST	`/api/auth/change-password`	Yes	Change password

Example: Login

```
POST /api/auth/login
{
  "username": "admin",
  "password": "Admin@123"
}

Response 200:
{
  "user": {
    "id": "abc123",
    "username": "admin",
    "displayName": "Super Admin",
    "role": "super_admin",
    "forcePasswordChange": false
  }
}
```

9.2 Assessment Endpoints

Method	Endpoint	Auth	Description
GET	`/api/assessments`	Yes	Get all assessments
GET	`/api/assessments/:id`	Yes	Get assessment details
POST	`/api/assessments`	Yes	Create assessment (upload XML)
PATCH	`/api/assessments/:id`	Yes	Update candidate info
DELETE	`/api/assessments/:id`	Yes	Delete assessment
POST	`/api/assessments/:id/complete`	Yes	Finalize assessment
POST	`/api/assessments/:id/report`	Yes	Generate PDF report
PATCH	`/api/assessments/:id/signatures`	Yes	Update signatures

Example: Create Assessment

```
POST /api/assessments
Content-Type: multipart/form-data

file: [XML file]
candidateName: "John Doe"
candidateEmail: "john@example.com"
candidateRole: "Solutions Architect"
assessorName: "Jane Smith"
assessmentDate: "2024-11-20"

Response 200:
{
  "assessment": {
    "id": "xyz789",
    "status": "analyzing",
    ...
  }
}
```

Example: Get Assessment Details

```
GET /api/assessments/xyz789

Response 200:
{
  "assessment": {
    "id": "xyz789",
    "candidateName": "John Doe",
    "status": "analyzed",
    "aiScore": 75.5,
    "manualScore": 0,
    "overallScore": 37.75,
    ...
  },
  "aiAnalyses": [
    {
      "categoryName": "Meta-model & Relationships",
      "score": 18.5,
      "maxScore": 25,
      "ragStatus": "amber",
      "issues": [...],
      ...
    }
  ],
  "manualScores": [
    {
      "id": "score1",
      "categoryName": "Meta-model & Relationships",
      "subCategoryName": "Element Correctness",
      "weight": 10,
      "rating": null,
      "weightedScore": 0,
      ...
    }
  ],
  "viewAnalyses": [...]
}
```

9.3 Manual Score Endpoints

Method	Endpoint	Auth	Description
PUT	`/api/manual-scores/:id`	Yes	Update rating/comments

Example: Update Manual Score

```
PUT /api/manual-scores/score1
{
  "rating": 4,
  "comments": "Good element usage, minor improvements needed"
}

Response 200:
{
  "score": {
    "id": "score1",
    "rating": 4,
    "weightedScore": 8.0,
    "comments": "Good element usage...",
    ...
  }
}
```

9.4 Admin Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/users`	Admin	Get all users
POST	`/api/admin/users`	Admin	Create new user
PATCH	`/api/admin/users/:id`	Admin	Update user info
DELETE	`/api/admin/users/:id`	Admin	Delete user
PATCH	`/api/admin/users/:id/toggle-active`	Admin	Enable/disable user
POST	`/api/admin/users/:id/reset-password`	Admin	Reset user password

Example: Create User

```
POST /api/admin/users
{
  "username": "jsmith",
  "displayName": "Jane Smith",
  "email": "jane@example.com"
}

Response 200:
{
  "user": {
    "id": "user123",
    "username": "jsmith",
    "displayName": "Jane Smith",
    "role": "assessor",
    "forcePasswordChange": true,
    ...
  },
  "temporaryPassword": "aB3$xY9#mK2p"  // Show once, never again
}
```

9.5 Utility Endpoints

Method	Endpoint	Auth	Description
GET	`/health`	No	Health check for Autoscale

10. Environment Variables and Configuration

10.1 Required Variables

Variable	Type	Example	Description
`DATABASE_URL`	String	`postgresql://user:pass@host/db`	PostgreSQL connection (auto-provided by Replit)
`SESSION_SECRET`	String	`min-32-random-chars-xyz123...`	Session encryption key (CRITICAL)
`OPENAI_API_KEY`	String	`sk-proj-abc123...`	OpenAI API key for AI analysis

10.2 Optional Variables

Variable	Default	Example	
PRODUCTION_DATABASE_URL	DATABASE_URL	postgresql://...	S p d
NODE_ENV	development	production	E n
PORT	5000	3000	S
OPENAI_MODEL	gpt-4o	gpt-4o-mini	C u
OPENAI_MAX_TOKENS	16000	8000	M A
OPENAI_TEMPERATURE	0	0.3	A ((
RESEND_API_KEY	-	re_abc123...	R A
SMTP_HOST	-	smtp.gmail.com	S (f
SMTP_PORT	587	465	S
SMTP_USER	-	user@gmail.com	S
SMTP_PASSWORD	-	app-password	S
SMTP_FROM	-	noreply@domain.com	E a
FRONTEND_URL	http://localhost:5000	https://app.replit.app	F f
REPLIT_DEPLOYMENT	-	1	A R p

10.3 Environment Variable Best Practices

Security: - Never commit `.env` file to git - Use Replit Secrets for production variables - Rotate `SESSION_SECRET` periodically - Keep API keys confidential

Validation:

```
// Critical check on startup
if (process.env.NODE_ENV === 'production' && !process.env.SESSION_SECRET)
  console.error('FATAL: SESSION_SECRET required in production');
  process.exit(1);
}
```

Example `.env` (Development):

```
# Database
DATABASE_URL=postgresql://user:password@localhost:5432/archimate_dev

# Authentication
SESSION_SECRET=development-secret-change-in-production-min-32-chars

# AI Analysis
OPENAI_API_KEY=sk-proj-your-key-here
OPENAI_MODEL=gpt-4o
OPENAI_MAX_TOKENS=16000

# Email (Optional)
RESEND_API_KEY=re_your_key_here
SMTP_FROM=noreply@yourdomain.com
FRONTEND_URL=http://localhost:5000

# Development
NODE_ENV=development
PORT=5000
```

10.4 Replit Secrets Configuration

How to add secrets on Replit: 1. Click “Tools” in sidebar 2. Select “Secrets” 3. Add key-value pairs for each variable 4. Secrets are automatically injected as environment variables

Required Production Secrets:

```
SESSION_SECRET=your-secure-random-string-min-32-chars
OPENAI_API_KEY=sk-proj-...
```

Optional Production Secrets:

```
PRODUCTION_DATABASE_URL=postgresql://...
RESEND_API_KEY=re_...
SMTP_FROM=noreply@yourdomain.com
FRONTEND_URL=https://your-app.replit.app
```

---

11. Testing and Quality Assurance

11.1 Manual Testing Walkthrough

11.1.1 Authentication Flow Test

Test Case 1: Super Admin First Login 1. Navigate to `/login` 2. Enter username: `admin`, password: `Admin@123` 3. Verify forced password change screen appears 4. Enter current password, new password, confirm 5. Verify redirect to dashboard 6. Logout and login with new password 7. Should login successfully

Test Case 2: Assessor Account Creation 1. Login as super admin 2. Navigate to `/admin` 3. Click “Create User” 4. Enter username, display name, email 5. Copy temporary password (shown once) 6. Logout 7. Login as new user with temporary password 8. Should force password change 9. Change password and verify access granted

11.1.2 Assessment Workflow Test

Test Case 3: Complete Assessment Workflow 1. Login as assessor 2. Navigate to `/` (New Assessment) 3. Upload valid ArchiMate XML file 4. Wait for AI analysis (status: analyzing → analyzed) 5. Verify blue “Next Steps” alert appears after AI completes 6. Navigate to Manual Assessment tab 7. Fill candidate information 8. Provide ratings for all 20 subcategories 9. Add comments (optional) 10. Draw and save assessor signature 11. Click “Complete Assessment” 12. Verify assessment marked as completed 13. Navigate to Detailed Report tab 14. Click “Download Report” 15. Verify PDF downloads with correct filename 16. Open PDF and verify all data present

Test Case 4: XML Upload Validation 1. Try uploading non-XML file (e.g., .txt, .pdf) 2. □ Should show error “Only XML files allowed” 3. Try uploading file >10MB 4. □ Should show error “File too large”

11.1.3 Signature Workflow Test

Test Case 5: Signature Persistence 1. In Manual Assessment, draw signature in pad 2. DO NOT click “Save Signature” 3. Refresh page 4. □ Signature should NOT appear (not saved) 5. Draw signature again 6. Click “Save Signature” button 7. Refresh page 8. □ Signature should persist 9. Generate PDF report 10. □ Signature should appear in PDF

11.1.4 Admin Functions Test

Test Case 6: User Management 1. Login as super admin 2. Create new assessor account 3. Toggle user active/inactive status 4. □ Inactive user should not be able to login 5. Reset user password 6. □ User should be forced to change password 7. Try deleting super admin account 8. □ Should show error “Cannot delete super admin” 9. Try deleting own account 10. □ Should show error “Cannot delete your own account”

11.2 Accessibility Testing

Keyboard Navigation: - [ ] Tab through all form fields - [ ] Tooltip help icons focusable and accessible - [ ] Enter key submits forms - [ ] Escape key closes modals - [ ] Focus indicators visible

Screen Reader Testing: - [ ] All form inputs have labels - [ ] Error messages announced - [ ] Status changes announced - [ ] Tooltip triggers have aria-labels

11.3 Cross-Browser Testing

Browsers to Test: - [ ] Chrome (latest) - [ ] Firefox (latest) - [ ] Safari (latest) - [ ] Edge (latest)

Features to Verify: - Login/logout - File upload (drag & drop) - Signature canvas (drawing) - PDF download - Table interactions

11.4 Performance Testing

Metrics to Monitor: - Page load time (<3 seconds) - API response time (<500ms average) - PDF generation time (<5 seconds) - AI analysis time (varies, 30-90 seconds)

Load Testing (recommended tools): - Artillery - k6 - Apache JMeter

11.5 Automated Testing Recommendations

Current State: Limited automated test coverage

Recommended Additions:

1. **Unit Tests** (Jest + React Testing Library):

```
// Example: Test rating calculation
describe('Manual Score Calculation', () => {
  it('calculates weighted score correctly', () => {
    const score = calculateWeightedScore(4, 10); // rating 4, weight 10
    expect(score).toBe(8.0); // (4/5) * 10 * 100 = 8.0
  });
});
```

2. **Integration Tests** (Supertest):

```
describe('POST /api/assessments', () => {
  it('creates assessment with valid XML', async () => {
    const res = await request(app)
      .post('/api/assessments')
      .attach('file', 'test-files/valid.xml')
      .field('candidateName', 'Test User')
      .expect(200);

    expect(res.body.assessment.status).toBe('analyzing');
  });
});
```

3. **E2E Tests** (Playwright):

```
test('complete assessment workflow', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[data-testid="input-username"]', 'admin');
  await page.fill('[data-testid="input-password"]', 'Admin@123');
  await page.click('[data-testid="button-login"]');

  // Upload file
  await page.goto('/');
  const fileInput = await page.locator('input[type="file"]');
  await fileInput.setInputFiles('test-files/sample.xml');
  await page.click('[data-testid="button-analyze"]');

  // Wait for AI analysis
  await page.waitForSelector('[data-testid="alert-next-steps"]');

  // Navigate to manual assessment
  await page.click('[data-testid="tab-manual"]');

  // Fill ratings and complete
  // ... continue workflow
});
```

---

12. Troubleshooting Common Issues

12.1 Authentication Issues

Issue: “Session cookie not persisting”

Symptoms: - User logged in but immediately logged out on refresh - “Not authenticated” errors on API calls

Root Cause: Missing `trust proxy` setting or incorrect secure cookie configuration

Solution:

```
// server/index.ts
app.set('trust proxy', 1); // CRITICAL for Replit Autoscale
```

Verification:

```
# Check logs for secure cookie warnings
grep -i "cookie" /tmp/logs/*.log
```

Issue: “SESSION_SECRET not set in production”

Symptoms: - Application crashes on startup with error message - Process exits immediately

Solution: 1. Add `SESSION_SECRET` to Replit Secrets 2. Ensure minimum 32 characters 3. Redeploy application

12.2 Database Issues

Issue: “Database connection timeout”

Symptoms: - 500 errors on all API calls - “Unable to connect to database” in logs

Solutions: 1. Verify `DATABASE_URL` is set correctly 2. Check Neon database status (may be paused) 3. Verify database allows connections from Replit IPs 4. Test connection: `bash npm run db:push # Should connect and sync schema`

Issue: “Migration fails with column type error”

Example Error:

```
Error: cannot alter type of column "id" from serial to varchar
```

Root Cause: Attempted to change primary key ID type

Solution: 1. **DO NOT change ID column types** 2. Check existing schema: `bash npm run db:introspect` 3. Match Drizzle schema to existing database structure 4. For drastic schema changes, create new table and migrate data

Issue: “Session table not found”

Symptoms: - Authentication works initially, then fails - “relation ‘session’ does not exist” error

Solution: Session table is auto-created by `connect-pg-simple`. Verify:

```
store: new pgSession({
  pool: db,
  tableName: 'session',
  createTableIfMissing: true  // Ensure this is true
})
```

12.3 OpenAI Integration Issues

Issue: “OpenAI API rate limit exceeded (429)”

Symptoms: - AI analysis fails with “QUOTA_EXCEEDED” error - Fallback analysis returned instead

Solutions: 1. Check OpenAI dashboard for usage limits 2. Upgrade OpenAI tier if needed 3. Implement request throttling 4. Consider using `gpt-4o-mini` for lower cost/faster limits

Temporary Workaround:

```
// Fallback analysis automatically triggered
// Manual assessment still possible
```

Issue: “OpenAI timeout on large files”

Symptoms: - Analysis takes >2 minutes - Network timeout errors

Solution: Already implemented - automatic file summarization:

```
// server/services/ai-analyzer.ts
if (estimatedTokens > 20000) {
  // Automatically summarizes large files
  // Reduces size by ~70%
}
```

Manual intervention if issue persists: 1. Ask user to export smaller model from Bizzdesign 2. Split model into multiple assessments 3. Increase OpenAI timeout (not recommended)

Issue: “OpenAI API key invalid”

Symptoms: - 401 Unauthorized errors - “Invalid API key” in logs

Solution: 1. Verify `OPENAI_API_KEY` in Replit Secrets 2. Check key starts with `sk-proj-` (new format) or `sk-` (old format) 3. Ensure key has not expired or been revoked 4. Test key with OpenAI Playground

12.4 File Upload Issues

Issue: “File upload fails with 413 Payload Too Large”

Symptoms: - Large XML files (>10MB) rejected - 413 HTTP status code

Solution: Current limit is 10MB. To increase:

```
// server/routes.ts
const upload = multer({
  limits: {
    fileSize: 20 * 1024 * 1024, // Increase to 20MB
  }
});
```

Consider: - Larger files = longer AI analysis time - Higher OpenAI costs - Potential timeout issues

Issue: “XML parsing fails”

Symptoms: - Upload succeeds but analysis fails - “Invalid XML format” error

Common Causes: 1. File is not valid XML (missing closing tags) 2. File has unsupported encoding (must be UTF-8) 3. File contains special characters not escaped

Solution:

```
// Validate XML before parsing
try {
  await parseXML(xmlContent);
} catch (error) {
  return res.status(400).json({
    message: 'Invalid XML file. Please check format and try again.'
  });
}
```

12.5 PDF Generation Issues

Issue: “Signature not appearing in PDF”

Root Cause: User drew signature but didn’t click “Save Signature” button

Solution: 1. Educate users: signature must be explicitly saved 2. Add tooltip: “Click ‘Save Signature’ after drawing” 3. Consider auto-save on signature pad blur (future enhancement)

Verification:

```
SELECT assessor_signature FROM assessments WHERE id = 'xxx';
-- Should return base64 PNG string, not null
```

Issue: “PDF download fails with 500 error”

Symptoms: - Error generating PDF - Server logs show PDFKit errors

Common Causes: 1. Missing data (null values in required fields) 2. Invalid base64

signature data 3. Memory limit exceeded for large reports

Solution:

```
// Add error handling in pdf-generator.ts
try {
  const pdfBuffer = await generatePDF(assessmentData);
} catch (error) {
  console.error('PDF generation failed:', error);
  // Return user-friendly error
}
```

12.6 Email Notification Issues

Issue: “Email not sent after finalization”

Symptoms: - Assessment completes successfully - No email received by candidate

Checklist: 1. Verify candidate email is provided 2. Check email service configured (Resend or SMTP) 3. Verify email credentials are correct 4. Check spam folder 5. Review server logs for email errors

Debug:

```
# Check logs for email service errors
grep -i "email" /tmp/logs/*.log
grep -i "resend" /tmp/logs/*.log
```

Test email service:

```
# Send test email via Resend API
curl -X POST https://api.resend.com/emails \
  -H "Authorization: Bearer ${RESEND_API_KEY}" \
  -H "Content-Type: application/json" \
  -d '{"from":"test@yourdomain.com","to":"test@example.com","subject":"Tes
```

12.7 Autoscale Deployment Issues

Issue: “Deployment succeeds but app doesn’t start”

Symptoms: - Build logs show success - Health check fails - “Service Unavailable” error

Debug Steps: 1. Check startup logs: `bash # View recent logs cat /tmp/logs/*.log | tail -100`

2. Verify environment variables set
3. Check database connectivity
4. Ensure port 5000 is bound correctly

Common Fixes:

```
// Ensure server listens on 0.0.0.0, not localhost
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server running on port ${PORT}`);
});
```

Issue: “Cold start delays”

Symptoms: - First request after idle takes 10-30 seconds - Subsequent requests fast

Expected Behavior: Autoscale pauses instances after inactivity

Mitigation: - Upgrade to always-on plan (if available) - Implement connection pooling - Use Neon serverless (auto-pause compatible)

12.8 Performance Issues

Issue: “Slow page load times”

Solutions: 1. Enable compression: `typescript import compression from 'compression'; app.use(compression());`

2. Optimize database queries:

```
// Use select only needed fields
const assessments = await db.select({
  id: assessments.id,
  candidateName: assessments.candidateName,
  status: assessments.status
}).from(assessments);
```

3. Implement caching for static data
4. Lazy load large components
5. Optimize images and assets

Issue: “Memory leaks”

Symptoms: - Application slows over time - Crashes after extended use - High memory usage in logs

Debug:

```
// Monitor memory usage
console.log('Memory usage:', process.memoryUsage());
```

Solutions: - Clear unused references - Properly close database connections - Avoid global state accumulation - Implement graceful shutdown

---

13. Future Enhancements

13.1 Recommended Improvements

High Priority

1. **Automated Testing**
   - Unit tests for core business logic
   - Integration tests for API endpoints
   - E2E tests for critical workflows
   - Target: 80% code coverage
2. **Enhanced Security**
   - Password complexity requirements
   - Rate limiting on authentication endpoints
   - Two-factor authentication (2FA) for admins
   - Session audit logging
   - Content Security Policy (CSP) headers
3. **Accessibility**
   - Full WCAG 2.1 AA compliance
   - Screen reader optimization
   - Keyboard navigation improvements
   - High contrast theme option
4. **Performance Optimization**
   - Database query optimization
   - Frontend bundle size reduction
   - Image optimization and lazy loading
   - CDN for static assets

Medium Priority

5. **User Experience**
   - Welcome modal/guided tour for first-time users
   - Inline help and documentation
   - Assessment templates
   - Bulk assessment import/export
   - Advanced filtering and search in records
6. **Reporting**
   - Customizable PDF templates
   - Excel export option
   - Dashboard analytics (total assessments, average scores, trends)
   - Comparative analysis (candidate vs. benchmark)
7. **AI Analysis**
   - Configurable AI model selection (GPT-4o, GPT-4o-mini, GPT-5)
   - Custom analysis prompts
   - AI confidence scoring
   - Historical trend analysis
8. **Collaboration**
   - Comments/notes on assessments
   - Assessment review workflow
   - Email notifications for status changes
   - Real-time collaboration (multiple assessors)

Low Priority

9. **Integration**
   - Bizzdesign API integration (direct export)
   - LDAP/Active Directory authentication
   - SSO (Single Sign-On) support
   - Webhook notifications
10. **Advanced Features**
    - Assessment versioning
    - Comparison between assessment versions
    - Custom scoring rubrics
    - Automated recommendations engine

13.2 Technical Debt

Areas Needing Refactoring:

1. **Error Handling**
   - Standardize error response format
   - Implement global error handler
   - Better client-side error recovery
2. **Code Organization**
   - Split large components (ManualAssessment, EnhancedAnalysisTabs)
   - Extract reusable hooks
   - Consistent naming conventions
3. **Type Safety**
   - Stricter TypeScript configurations
   - Eliminate `any` types
   - Add runtime type validation
4. **Testing Infrastructure**
   - Set up CI/CD pipeline
   - Automated test runs on deployment
   - Performance regression testing

13.3 Scalability Considerations

Current Limitations: - Single database instance - No caching layer - Synchronous AI analysis (blocks during processing)

Scaling Strategy:

1. **Database**:
   - Read replicas for analytics
   - Connection pooling optimization
   - Database sharding (if >1M assessments)
2. **Caching**:
   - Redis for session storage
   - Cache static assessment data
   - CDN for frontend assets
3. **Background Jobs**:
   - Message queue (BullMQ, RabbitMQ)
   - Separate AI analysis workers
   - Parallel processing for bulk operations
4. **Monitoring**:
   - Application performance monitoring (APM)
   - Error tracking (Sentry)
   - Usage analytics
   - Cost monitoring (OpenAI, database)

13.4 Compliance and Governance

Data Privacy: - [ ] GDPR compliance audit - [ ] Data retention policies - [ ] Right to be forgotten implementation - [ ] Data export functionality - [ ] Privacy policy and terms of service

Audit Trail: - [ ] User action logging - [ ] Assessment change history - [ ] Login attempt tracking - [ ] Admin action audit log

13.5 Documentation Needs

Missing Documentation: 1. API reference with OpenAPI/Swagger spec 2. Database schema documentation with ER diagrams 3. Deployment runbook with rollback procedures 4. User manual with screenshots 5. Video tutorials for common workflows 6. FAQ and knowledge base

---

Appendix A: Glossary

ArchiMate: Enterprise architecture modeling language and standard (version 3.2 used)

Assessment: Complete evaluation of an ArchiMate model, combining AI and manual analysis

Assessor: User role with permission to create and complete assessments

RAG Status: Red/Amber/Green indicator of issue severity in AI analysis

Manual Score: Human expert evaluation score (0-5 rating scale)

Weighted Score: Calculated score accounting for subcategory importance

Overall Score: Average of AI score and manual score

Finalization: Locking assessment after completion, making it immutable

Super Admin: User role with full system access including user management

---

Appendix B: Quick Reference Commands

```
# Development
npm install                    # Install dependencies
npm run dev                    # Start development server
npm run build                  # Build for production
npm start                      # Start production server

# Database
npm run db:push                # Push schema changes to database
npm run db:push --force        # Force push (ignores warnings)
npm run db:studio              # Open Drizzle Studio (GUI)

# Deployment
git push                       # Auto-deploys on Replit (if connected)

# Debugging
tail -f /tmp/logs/*.log        # Monitor application logs
grep "ERROR" /tmp/logs/*.log   # Find errors in logs
```

Appendix C: Support and Maintenance

Handover Contacts

Original Development Team: [Contact information]
OpenAI Support: https://help.openai.com
Replit Support: https://replit.com/support
Neon Database Support: https://neon.tech/docs

Useful Resources

- **ArchiMate 3.2 Specification**: https://pubs.opengroup.org/architecture/archimate32-doc/
- **Drizzle ORM Docs**: https://orm.drizzle.team/
- **TanStack Query Docs**: https://tanstack.com/query/latest
- **Shadcn/UI Components**: https://ui.shadcn.com/
- **Replit Autoscale Docs**: Search Replit documentation for Autoscale

Regular Maintenance Tasks

Daily: - [ ] Monitor error logs - [ ] Check OpenAI usage and costs - [ ] Verify health check endpoint

Weekly: - [ ] Review failed assessments - [ ] Check database storage usage - [ ] Monitor user feedback

Monthly: - [ ] Review security updates - [ ] Update dependencies - [ ] Backup database - [ ] Review and rotate API keys

Quarterly: - [ ] Performance audit - [ ] Security audit - [ ] User access review - [ ] Cost optimization review

---

Document Version History

Version	Date	Author	Changes
1.0.0	2024-11-20	Development Team	Initial solution documentation for handover

---

End of Solution Documentation

For questions or clarifications, please refer to the codebase comments or contact the development team.